

Project Documentation: Academic Records System

Name: S'fezekile Chandu

Project Overview

Problem Statement

Eduvos Institute of Higher Education required a robust MySQL-based academic records system to address critical challenges in their academic data management:

1. **Invalid Registrations:** Students being added to exams without official module registration ("ghost entries")
2. **Daily Performance Tracking:** Lack of systematic tracking of student performance metrics
3. **Concurrency Issues:** Multiple users accessing the system simultaneously causing data conflicts
4. **Unauthorized Changes:** Exam mark edits without audit trails

Solution Approach

Developed a comprehensive database system with:

- Data validation constraints
- Automated performance tracking
- Transaction management for concurrency control
- Audit trails for security
- Updatable views for simplified data access

Technology Stack

- **Database:** MySQL
- **Concepts:** Stored Procedures, Functions, Triggers, Views, Transaction Management

Database Schema Design

Tables Structure

Student Table

```
CREATE TABLE student (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(45) NOT NULL,  
    surname VARCHAR(45) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL CHECK (email LIKE "%@%.%"),  
    phone VARCHAR(45) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    registration_date DATE NOT NULL  
);
```

Module Table

```
CREATE TABLE module (  
    module_id INT AUTO_INCREMENT PRIMARY KEY,  
    module_name VARCHAR(100) NOT NULL,  
    module_code VARCHAR(20) UNIQUE NOT NULL,  
    credits INT NOT NULL CHECK (credits > 0)  
);
```

Exam Table

```
CREATE TABLE exam (  
    exam_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_id INT NOT NULL,  
    module_id INT NOT NULL,  
    exam_date DATE NOT NULL,  
    total_mark DECIMAL(5,2) CHECK (total_mark >= 0 AND total_mark <= 100),  
    FOREIGN KEY (student_id) REFERENCES student(student_id),  
    FOREIGN KEY (module_id) REFERENCES module(module_id)  
);
```

Module Stats Table

```
CREATE TABLE module_stats (  
    stats_id INT AUTO_INCREMENT PRIMARY KEY,  
    module_id INT NOT NULL,  
    stats_date DATE NOT NULL,  
    total_attempts INT DEFAULT 0,  
    distinctions INT DEFAULT 0,  
    passes INT DEFAULT 0,  
    fails INT DEFAULT 0,  
    FOREIGN KEY (module_id) REFERENCES module(module_id)  
);
```

Exam Audit Table

```
CREATE TABLE exam_audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    exam_id INT NOT NULL,  
    old_total_mark DECIMAL(5,2),  
    new_total_mark DECIMAL(5,2),  
    changed_by VARCHAR(100),  
    change_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (exam_id) REFERENCES exam(exam_id)  
);
```

Top Performers Table

```
CREATE TABLE top_performers (  
    top_performer_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_id INT NOT NULL,  
    module_id INT NOT NULL,  
    exam_date DATE NOT NULL,  
    total_mark DECIMAL(5,2) NOT NULL,  
    recorded_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (student_id) REFERENCES student(student_id),  
    FOREIGN KEY (module_id) REFERENCES module(module_id)  
);
```

Stored Procedures and Functions

1. sp_Populate_Module_Stats()

Purpose: Calculate and store module statistics using cursor

```
DELIMITER //
CREATE PROCEDURE sp_Populate_Module_Stats()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE module_id_var INT;
    DECLARE total_attempts_var INT;
    DECLARE distinctions_var INT;
    DECLARE passes_var INT;
    DECLARE fails_var INT;

    DECLARE module_cursor CURSOR FOR
        SELECT module_id FROM module;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN module_cursor;

    module_loop: LOOP
        FETCH module_cursor INTO module_id_var;
        IF done THEN
            LEAVE module_loop;
        END IF;

        -- Calculate statistics for each module
        SELECT COUNT(*) INTO total_attempts_var
        FROM exam WHERE module_id = module_id_var;

        SELECT COUNT(*) INTO distinctions_var
        FROM exam WHERE module_id = module_id_var AND total_mark >=
80;

        SELECT COUNT(*) INTO passes_var
        FROM exam WHERE module_id = module_id_var AND total_mark >=
50;

        SELECT COUNT(*) INTO fails_var
```

```

        FROM exam WHERE module_id = module_id_var AND total_mark <
50;

        -- Insert or update module_stats
        INSERT INTO module_stats (module_id, stats_date,
total_attempts, distinctions, passes, fails)
        VALUES (module_id_var, CURDATE(), total_attempts_var,
distinctions_var, passes_var, fails_var)
        ON DUPLICATE KEY UPDATE
            total_attempts = total_attempts_var,
            distinctions = distinctions_var,
            passes = passes_var,
            fails = fails_var;

    END LOOP;

    CLOSE module_cursor;
END //
DELIMITER ;

```

2. sp_Exam_Stats()

Purpose: Return summary report of exam statistics

```

DELIMITER //
CREATE PROCEDURE sp_Exam_Stats(OUT total_students INT, OUT
total_distinctions INT, OUT total_passes INT, OUT total_fails INT)
BEGIN
    SELECT COUNT(*) INTO total_students FROM exam;
    SELECT COUNT(*) INTO total_distinctions FROM exam WHERE
total_mark >= 80;
    SELECT COUNT(*) INTO total_passes FROM exam WHERE total_mark >=
50 AND total_mark < 80;
    SELECT COUNT(*) INTO total_fails FROM exam WHERE total_mark < 50;
END //
DELIMITER ;

```

3. sp_Insert_Exam_With_Validation()

Purpose: Safely insert exam records with transaction management

```

DELIMITER //
CREATE PROCEDURE sp_Insert_Exam_With_Validation(
    IN p_student_id INT,
    IN p_module_id INT,
    IN p_exam_date DATE,
    IN p_total_mark DECIMAL(5,2)
)
BEGIN
    DECLARE enrollment_exists INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    -- Set transaction isolation level
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;

    -- Check if student is enrolled in the module
    SELECT COUNT(*) INTO enrollment_exists
    FROM student_module
    WHERE student_id = p_student_id AND module_id = p_module_id;

    IF enrollment_exists = 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student not
enrolled in the specified module';
    ELSE
        -- Create savepoint
        SAVEPOINT before_exam_insert;

        -- Insert exam record
        INSERT INTO exam (student_id, module_id, exam_date,
total_mark)
        VALUES (p_student_id, p_module_id, p_exam_date,
p_total_mark);

        -- Update module statistics
        CALL sp_Populate_Module_Stats();

        COMMIT;
    END IF;
END //

```

```
DELIMITER ;
```

4. sp_Distinctions_Per_Module()

Purpose: Show number of distinctions per module

```
DELIMITER //
CREATE PROCEDURE sp_Distinctions_Per_Module()
BEGIN
    SELECT
        m.module_name,
        COUNT(*) as distinction_count
    FROM exam e
    JOIN module m ON e.module_id = m.module_id
    WHERE e.total_mark >= 80
    GROUP BY m.module_id, m.module_name
    ORDER BY distinction_count DESC;
END //
DELIMITER ;
```

5. fn_Top_Student_Last30Days()

Purpose: Return best-performing student over past 30 days

```
DELIMITER //
CREATE FUNCTION fn_Top_Student_Last30Days()
RETURNS VARCHAR(200)
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE top_student_name VARCHAR(200);
    DECLARE top_avg_mark DECIMAL(5,2);

    SELECT
        CONCAT(s.first_name, ' ', s.surname),
        AVG(e.total_mark)
    INTO top_student_name, top_avg_mark
    FROM exam e
    JOIN student s ON e.student_id = s.student_id
    WHERE e.exam_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
```



```
GROUP BY e.student_id
ORDER BY AVG(e.total_mark) DESC
LIMIT 1;

RETURN CONCAT(top_student_name, ' - Average: ',
COALESCE(top_avg_mark, 0));
END //
DELIMITER ;
```

Triggers Implementation

1. trg_exam_before_update

Purpose: Log mark changes for audit trail

```
DELIMITER //
CREATE TRIGGER trg_exam_before_update
BEFORE UPDATE ON exam
FOR EACH ROW
BEGIN
    IF OLD.total_mark != NEW.total_mark THEN
        INSERT INTO exam_audit (exam_id, old_total_mark,
new_total_mark, changed_by)
        VALUES (OLD.exam_id, OLD.total_mark, NEW.total_mark,
CURRENT_USER());
    END IF;
END //
DELIMITER ;
```

2. trg_exam_after_insert

Purpose: Log high scores in top_performers table

```
DELIMITER //
CREATE TRIGGER trg_exam_after_insert
AFTER INSERT ON exam
FOR EACH ROW
```

```
BEGIN
    IF NEW.total_mark > 90 THEN
        INSERT INTO top_performers (student_id, module_id, exam_date,
total_mark)
            VALUES (NEW.student_id, NEW.module_id, NEW.exam_date,
NEW.total_mark);
    END IF;
END //
DELIMITER ;
```

Views Creation

1. vw_status - Student Status View

```
CREATE VIEW vw_status AS
SELECT
    e.student_id,
    e.module_id,
    m.module_name,
    s.first_name,
    s.surname,
    e.total_mark,
    CASE
        WHEN e.total_mark >= 80 THEN 'Distinction'
        WHEN e.total_mark >= 50 THEN 'Pass'
        ELSE 'Fail'
    END AS status
FROM exam e
JOIN student s ON s.student_id = e.student_id
JOIN module m ON m.module_id = e.module_id;
```

2. vw_avg_marks_by_city - Performance by City

```
CREATE VIEW vw_avg_marks_by_city AS
SELECT
    s.city,
    COUNT(e.exam_id) as total_exams,
```

```
    AVG(e.total_mark) as average_mark,  
    MAX(e.total_mark) as highest_mark,  
    MIN(e.total_mark) as lowest_mark  
FROM student s  
LEFT JOIN exam e ON s.student_id = e.student_id  
GROUP BY s.city  
ORDER BY average_mark DESC;
```

Sample Data and Testing

Sample Data Insertion

```
-- Insert sample students  
INSERT INTO student (first_name, surname, email, phone, city,  
registration_date) VALUES  
( 'John', 'Smith', 'john.smith@student.eduvos.ac.za', '0711234567',  
'Johannesburg', '2024-01-15'),  
( 'Sarah', 'Johnson', 'sarah.johnson@student.eduvos.ac.za',  
'0722345678', 'Cape Town', '2024-01-20'),  
( 'David', 'Brown', 'david.brown@student.eduvos.ac.za', '0733456789',  
'Durban', '2024-02-01');
```

```
-- Insert sample modules  
INSERT INTO module (module_name, module_code, credits) VALUES  
( 'Database Management', 'ITMDA0-33', 12),  
( 'Programming Fundamentals', 'ITPRG0-44', 12),  
( 'Web Development', 'ITWEB0-55', 10);
```

```
-- Insert sample exam records  
INSERT INTO exam (student_id, module_id, exam_date, total_mark)  
VALUES  
(1, 1, '2024-06-15', 85.5),  
(1, 2, '2024-06-20', 72.0),  
(2, 1, '2024-06-15', 91.0),  
(2, 3, '2024-06-25', 68.5),  
(3, 1, '2024-06-15', 45.0);
```

Testing Procedures and Functions

```
-- Test exam insertion with validation
CALL sp_Insert_Exam_With_Validation(1, 1, '2024-07-10', 78.5);

-- Test module statistics
CALL sp_Populate_Module_Stats();

-- Test exam statistics
CALL sp_Exam_Stats(@total, @distinctions, @passes, @fails);
SELECT @total, @distinctions, @passes, @fails;

-- Test distinctions per module
CALL sp_Distinctions_Per_Module();

-- Test top student function
SELECT fn_Top_Student_Last30Days();
```

Database Backup

Backup Creation

The database backup was created using MySQL dump utility:

```
mysqldump -u username -p eduvos > eduvos_backup.sql
```

Backup Contents

- Complete database schema with all tables
- Stored procedures, functions, triggers, and views
- Sample data for testing
- User permissions and constraints

Conclusion

Project Achievements

1. **Eliminated Ghost Entries:** Implemented enrollment validation in exam insertion procedures
2. **Daily Performance Tracking:** Automated statistics calculation and storage
3. **Concurrency Control:** Proper transaction management with REPEATABLE READ isolation level
4. **Audit Trail:** Comprehensive logging of all exam mark changes
5. **Top Performers Tracking:** Automatic identification of exceptional students
6. **Simplified Data Access:** Updatable views for common reporting needs

Technical Implementation Success

- All database objects created according to specifications
- Proper constraint implementation for data integrity
- Comprehensive error handling in stored procedures
- Efficient cursor usage for batch processing
- Proper transaction management for data consistency

Learning Outcomes

- Advanced MySQL programming concepts
- Database design and optimization
- Transaction management and concurrency control
- Stored procedure and trigger development
- Database security and audit trail implementation

End of Documentation